.NET frontend developer AvaloniaUI

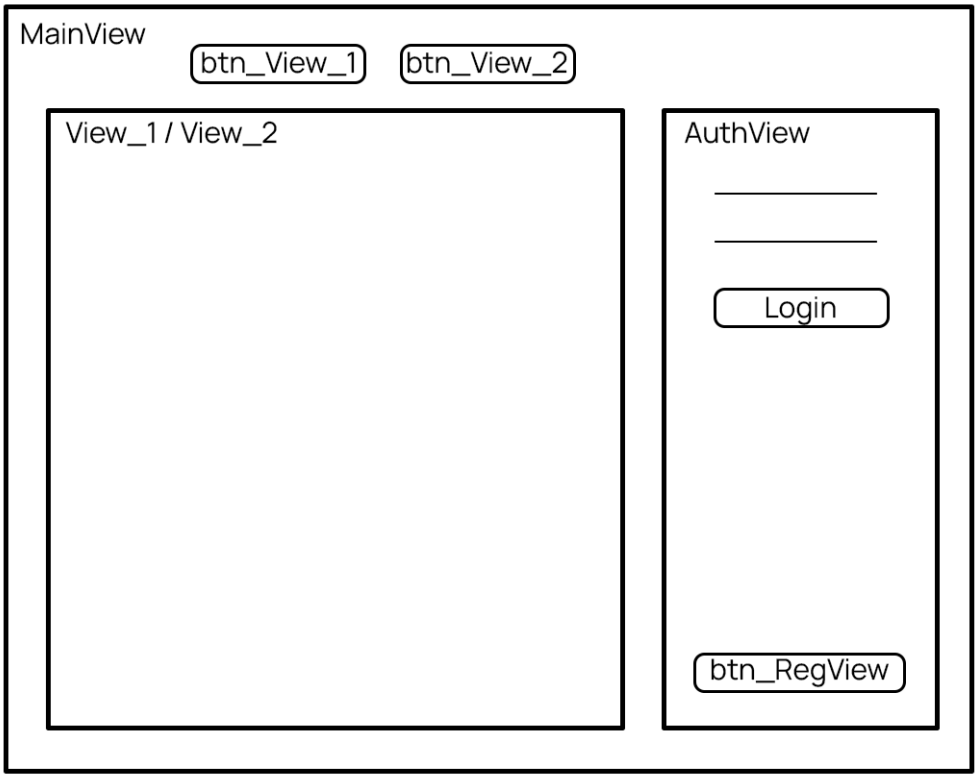
Разработать Avalonia UI приложение .NET 10 с паттерном MVVM.

Цель — продемонстрировать навыки проектирования MVVM-приложения, работы с REST API и организации кода.

Требования:

- Реализовать Avalonia UI .NET 10 приложение с архитектурой MVVM
- Использовать Community Toolkit
- Авторизация и регистрация через API
- Асинхронные выполнение запросов
- Использовать внедрение зависимостей
- Приветствуется базовое оформление UI и стили

Важно показать: чистую структуру кода, разделение ответственности, работу с асинхронными запросами и базовые навыки проектирования UI.



Структура приложения

1. Описание:

Главная страница приложения (**MainView**) содержит в себе:

- Меню/кнопки которые позволяют выбрать активный раздел приложения (**View_1** или **View_2**). Учесть возможность добавления новых разделов.
- Блок для отображения активного раздела **View_1/View_2**
- Блок для отображения **AuthView**

AuthView (пользователь не авторизован):

- Форма авторизации: email, пароль, кнопка Login
- Кнопка регистрации, по нажатию на которую открывается новое диалоговое окно. В окне указываются данные пользователя: email, имя и пароль, а также кнопка “Зарегистрироваться”. Автоматическая авторизация после регистрации.

AuthView (пользователь авторизован):

- Отображается кнопка разлогина

View_1 (пользователь не авторизован):

- Текст с просьбой авторизоваться или зарегистрироваться

View_1 (пользователь авторизован):

- Отображаются данные пользователя:
  - id
  - email
  - имя
  - аватар
  - роль

View_2:

- Укажите свои данные как о разработчике приложения

Навигация между разделами должна быть реализована через MVVM (без обращения к UI из кода представления).

2. API

API для выполнения запросов: https://fakeapifordevs.vercel.app/docs

Используемые методы:

- POST /api/auth/register — регистрация;
- POST /api/auth/login — авторизация;
- GET /api/auth/me — получение данных текущего пользователя;
- POST /api/auth/logout — завершение сессии.

Состояние авторизации должно корректно отражаться в UI (**AuthView / View_1 / View_2**).

3. Заметки

- Иметь обоснования для всех используемых nuget пакетов
- Внедрение зависимостей можно реализовать с использованием простого контейнера (например, Microsoft.Extensions.DependencyInjection).
- Минимальное оформление интерфейса приветствуется (простая структура, базовые стили, аккуратная компоновка). Главное — читаемость и понятная организация XAML.

4. При оценке решения учитывается:

- архитектура проекта (разделение View / ViewModel / Model)
- читаемость и чистота кода
- корректность работы с API
- обработка ошибок и асинхронность
- обоснование NuGet-пакетов
- аккуратность XAML и соблюдение MVVM